TRACE-RPG: Auditable Neuro-Symbolic Commit Gates for Interactive Game Worlds

Anonymous Author(s)

***Abstract*—LLMs can generate expressive quests and dialogue, but fluent/schema-valid text does not show that an event is executable, authorized, or consistent with canonical game state. We present TRACE-RPG, a symbolic commit gate treating events as untrusted transaction proposals. A type-strict parser, external action policy, and seven deterministic checks in six state-relative families decide transitions; failed candidates receive at most $K + 1$ recorded repair attempts, and passing candidates are revalidated before mutation. The validator-error-guided operator ρ reads only the prior candidate and typed validation errors (its sole state-derived input) and never accesses authoritative `WorldState` directly. Unkeyed SHA-256 content links, state-semantic replay, and assignment-complete accounting audit terminal outcomes. Four separate lanes cover offline conformance on one authored world, single-model live screening, closed-world knowledge-graph link simulation without runtime authority, and a participant-free Godot engineering slice. Offline evidence is mechanism-only. At $K{=}1$, live screening observed guided 5/5 versus blind 0/5 commits only in a deliberately repairable policy-blind regime and remains pilot-only. No lane establishes population efficacy, model superiority, player experience, or commercial-engine performance.**

***Index Terms*—game AI, interactive narrative, symbolic validation, constrained generation, reproducibility**

I. Introduction

INTERACTIVE generation can require absent items, assert facts unknown to the speaker, disclose forbidden quest information, introduce unauthorized effects, or regress quest stages. Executable text environments distinguish plausible utterances from valid transitions [1], [2]; grounded dialogue and personalized quests demonstrate useful quest, entity, and graph context [3], [4], not mutation authority. **TRACE-RPG treats generated events as untrusted transaction proposals; only an external deterministic commit gate permits canonical mutation** (Fig. 1).

Grammar-constrained decoding, query languages, and incremental parsers constrain structure [5], [6], [7], not truth in the current world. Retrieval and memory supply evidence [8], [9], [10], not state-transition authority.

The auditable transaction protocol contributes **C1—Contracts:** versioned trust-boundary contracts/parser (Sec. IV; E1, ENG1); **C2—Controller:** seven checks in six state-relative families, bounded repair, unchanged fallback, and pre-commit revalidation (Sec. IV-C; E1, ENG1); **C3—Audit:** SHA-256-linked records, state-semantic replay, and episode continuity (Sec. IV-D; E1, ENG1); **C4—Harness:** assignment-complete offline conformance retaining classified terminal rows even without proposal traces (Sec. V; E1); and **C5—Repair:** typed $\rho(a, E)$ versus blind retry/state-reading oracle (Sec. IV-C; E1, E2). Fixed-goal NPC operation adds descriptive NPC-ENG1/NPC-SCR1 evidence, not C6 or C-RESULT; Sec. V separates its ownership, counts, role gates, provenance, core Python–hard-policy, and normal-play boundaries. Machine-checked Tables I and IV map C1–C5 to prior topics, lanes, and inference ceilings.

TABLE I
CONTRIBUTIONS, PRIOR TOPICS, EVIDENCE LANES, AND INFERENCE CEILINGS (GENERATED FROM THE STRUCTURED MATRICES)

ID	Contribution	Prior topics (refs)	Lanes	Inference ceiling
C1	Trust-boundary contracts	T1, T2, T3 (17)	E1, ENG1	encoded-field contract conformance
C2	Validate–repair–commit controller	T1, T8 (14)	E1, ENG1	mechanism behavior on frozen cases
C3	Audit-linked evidence layer	T4, T7, T8 (6)	E1, ENG1	integrity/replay on named fixtures
C4	Assignment-complete harness	T1, T4, T6, T7 (20)	E1	exact designed-case accounting
C5	Counterexample-guided operator ρ	T1, T8, T9 (8)	E1, E2	frozen classes + one regime screen

T1 = grounded game worlds; T2 = retrieval/memory agents; T3 = structured generation; T4 = environments/benchmarks; T5 = player experience; T6 = human/LLM evaluation; T7 = statistics/reproducibility; T8 = planning/runtime interposition; T9 = feedback repair. Lanes are defined in Table IV. All rows are authored-fixture verified; none supports an efficacy claim.

These lanes are non-interchangeable: Table IV and Sec. V fix their separate denominators, provenance, and inference ceilings; none supports population efficacy, model ranking, player benefit, retrieval benefit, or C6.

II. Related Work and Research Gap

A. Interactive Narrative and Game Agents

KNUDGE grounds branching NPC dialogue in quest/entity specifications [3]; knowledge-graph quests [4], memory–reflection–planning [11], and trainable role-playing agents [12] address complementary grounding and believable behavior. Studies of player preference/perceived dialogue quality [13], [14], [15] and a 2025 preprint's role-sensitive stability–improvisation trade-offs [16] motivate separating hard admissibility from optional naturalness/player-experience constructs.

Experience-driven generation, observer-labelled engagement, and limits of vision-language engagement inference motivate future affect work [17], [18], [19]; affect is non-authoritative and absent here.

Manuscript prepared for anonymous review.

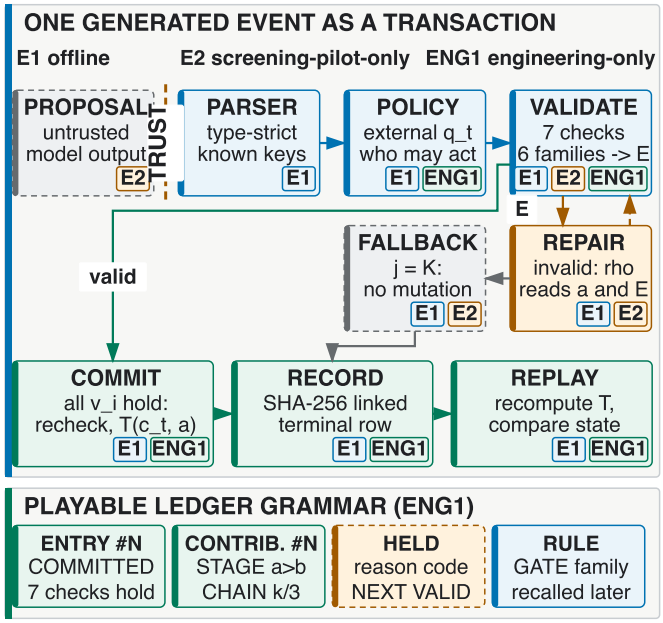


Fig. 1. One generated event as a transaction. A typed candidate crosses the trust boundary; seven checks in six state-relative families consult external policy and c_t. Typed errors E feed bounded ρ (dashed) or unchanged fallback; passing candidates are rechecked, committed, recorded, and replayed. Tags mark measurement scope, not claims: the core path is E1/E2, legacy ENG1 only; additive NPC `npc-goal-recovery/1.1.0` is a distinct Godot-authoritative intent/planning path, not a demonstrated core Python–hard-policy round trip. The lower strip is the committed-snapshot ledger grammar.

B. Environments and Benchmarks

TextWorld and ScienceWorld expose explicit interactive state [1], [2], MineDojo adds open-ended embodied tasks [20], and GVGAI separates agent, game, and content tracks [21]. AgentBench, AgentBoard, SOTOPIA, and BALROG evaluate agents across heterogeneous, process-diagnosed, social, and long-horizon game settings [22], [23], [24], [25]; automated play-style agents complement rather than replace human evaluation [26]. Across these reviewed environments and benchmarks, we did not identify a single protocol that combines transactional authorization, typed repair, semantic replay, and assignment-complete accounting; this is a scope-limited review finding, not a first-ever claim. NCP-Bench adds a long-horizon interactive-narrative benchmark for commitment and fact consistency, but it does not evaluate TRACE-RPG or player outcomes [27].

C. Constraints, Neuro-Symbolic Validation, and Retrieval

Constrained decoders improve structure [5], [6], [7]; TRACE-RPG requires independent state-relative authorization. Generate-and-verify precedents include LLM-Modulo's model-proposed plans/external sound critics [28] and SayCan's state-dependent skill-affordance checks [29]. AgentSpec's trigger–predicate–enforcement rules [30] contrast with our game-state predicates, repair contract, and replayable records. Setting the DC grounds D&D agents in precondition-checked tool calls [31]; we gate free-text proposals after parsing/external policy evaluation. STORY2GAME's LLM generates its own action preconditions/effects [32]; our independently authored policy drives a deterministic critic returning typed counterexamples. PANGeA uses LLM judgement/refinement against designer rules [33]; IVIE incrementally generates interactive-fiction worlds with symbolic validation [34]; its 2026 successor restricts LLMs to authored world-state transformations for consistency by construction [35]. Transaction, audit-record, and repair-operator questions remain separate evidence targets. None implies superiority: validity covers only encoded predicates. JSONSchemaBench measures structural constrained-output compliance, coverage, and quality; those measures do not establish state-relative semantic authorization [36].

G-Retriever, HippoRAG, and Mem0 address subgraph retrieval/long-term memory [8], [9], [10]; Generative Agents and Character-LLM address behavioral memory/persona [11], [12]. Such context has no commit authority here; benefits are unmeasured.

D. Auditability and Reproducibility

Process metrics, sparse-run caution, reproducibility, and environmental reporting motivate failure retention and frozen artifacts [23], [37], [38], [39]. Human/LLM evaluation and future participant analyses require construct/rater/bias controls, justified margins, and random-effects structures [40], [41], [42], [43], [44], [45]. Record-and-replay supports experience reuse [46]; TRACE-RPG claims only semantic replay of admitted transitions. Leakage checks protect frozen denominators but do not create efficacy evidence [47].

Novelty lies in the implemented combination: runtime enforcement [30], symbolic game-state management [31], [32], feedback repair [48], record/replay [46], action interposition [49], classical preconditions/effects [50], permitted-transition enforcement [51], transaction abort [52], authored social logic [53], and belief-aware narrative planning [54] have precedents. C1–C5 combine independent contracts, validate–repair–commit, linked semantic replay/continuity, complete accounting, and typed-error repair at the game-state commit boundary. Absence from a report does not imply absence from an implementation.

III. Problem Definition and Guarantee Boundary

Validation input at step t is $c_t = (G_t, q_t)$: typed snapshot fields G_t and externally authored action-policy, disclosure, and quest-stage constraints q_t stored with them. Terminal history $h_{<t}$ is separate from `WorldState`; predicates never read it, nor is state reconstructed from logs. An accepted one-step transition updates G_t under unchanged q_t; policy-version rotation is out of scope. Retrieval, narrative scores, model rationales, memory summaries, and future affect estimates are non-authoritative proposal context.

An untrusted proposer emits candidate a_t. Strict parsing first enforces the shared candidate-key contract, including rejection of unknown top-level fields, and establishes the declared schema. The validator returns

$$V(c_t, a_t) = (v_{\text{policy}}, v_{\text{pre}}, v_{\text{reach}}, v_{\text{know}}, v_{\text{disc}}, v_{\text{elig}}, v_{\text{stage}}, E_t), \quad (1)$$

TABLE II
ENCODED VALIDATOR BOUNDARY

Predicate	Rejects	On failure
Action policy	unknown action/type	unchanged
Precondition	absent/false requirement	unchanged
Reachability	inaccessible object	unchanged
NPC knowledge	undeclared known fact	unchanged
Disclosure	forbidden fact	unchanged
Quest eligibility	ineligible quest action	unchanged
Quest stage	regressive or unauthorized stage	unchanged

where E_t is a structured error set. Mutation is

$$c_{t+1} = \begin{cases} T(c_t, a_t), & \bigwedge_{i=1}^{7} v_i = 1, \\ c_t, & \text{otherwise.} \end{cases} \tag{2}$$

The controller asserts the following implementation invariants under its encoded contracts; the pilot tests their mechanisms with deterministic valid, invalid, repair, fallback, replay, and fault-injection fixtures rather than presenting general theorems. **I1 (exhausted-failure immutability):** if no returned candidate through budget K validates, or proposal/controller execution fails before commit, the returned state equals the supplied prior state. **I2 (bounded recorded attempts):** provided every proposal and repair callback returns, repair budget $K \geq 0$ admits at most $K + 1$ recorded candidate attempts; a successful commit adds one defensive validation immediately before mutation. The runtime does not enforce callback deadlines, so I2 is not a wall-clock termination guarantee. **I3 (effect authorization):** an accepted candidate contains no declared fact effect in `CandidateAction.effects` outside `ActionPolicy.allowed_effects`, and any quest-stage target is explicitly authorized by that policy. **I4 (state-semantic replay):** with the frozen schemas and deterministic transitions, an accepted trace reconstructs its stated terminal state.

These invariants assume correct policy authorship, faithful extraction into declared candidate fields, deterministic software versions, returning callbacks, and single-process file ownership. Replay revalidates the recorded structured candidate and transition; it does *not* verify the provenance, prompts, or stochastic generation of intermediate repairs, and natural-language implications omitted from structured fields remain outside the guarantee.

IV. TRACE-RPG ARCHITECTURE AND ALGORITHMS

A. Trust Boundary and Contracts

The model/recorded adapter lies outside the canonical owner (Fig. 1). Parsing rejects ambiguous numeric/collection types and unknown top-level fields; separate action policies define required preconditions/allowed effects. The versioned bridge carries observations, proposals, validations, rejections, fallbacks, and commits; only valid commits authorize mutation. Interface separation demonstrates neither commercial-engine portability nor proposal-default/replay-parser equivalence: evidence supports unknown-key rejection parity only.

B. Fixed-goal NPC integration

The NPC lane uses `npc-goal-recovery/1.1.0` and a mechanically exact generated runtime mirror. Config, disclosure-policy, and `project.godot` hashes are recorded by SHA-256 prefixes `d63f5679f360`, `af0a1075294b`, and `2e7966bf2ac6`; full digests are in `research/academic-pipeline/npc-goal-recovery/verified-run-006/`. Before writing, Godot synchronously checks session/epoch/request/scenario/NPC/goal/version, policy/config hashes, revision/state hash, deadline/budget, and issued-action binding. The service listens on `127.0.0.1:8768`; browser origins are `localhost` and `127.0.0.1:8877`, not 8767. Codex is the non-authoritative OAuth interpreter that proposes typed intent; Python validates and verifies typed claims and computes the public-projection plan; Godot alone authorizes and writes canonical state. P has no rule fallback. The fixed harbor-signal goal is version 1 for `captain_mira`; no utterance or model output may replace it. Utterance requests commit raw UTF-8 bytes via `raw_utterance_sha256`; observation-only requests carry `utterance:null` and a null digest, checked before interpretation. The closed loop is proposal $\rightarrow$ verification $\rightarrow$ public-projection BFS plan $\rightarrow$ player/NPC role gate $\rightarrow$ actual result $\rightarrow$ observation-only replan. Python validates and verifies typed claims from public flags; BFS emits only the enabled first step toward the fixed goal and never reads hidden state. `acquire_object`/`install_lens` are `player_request`; `reveal_hint` is Mira-only `commit_world` when known and disclosure-authorized; `clarify`/`negotiate` are dialogue-only. No public route yields `NO_ROUTE` and no physical step. Replanning uses only the returned observation/state reference, not the prior utterance or a changed goal. Episode budgets and deadlines persist across utterance epochs; intent calls, action attempts, replans, and BFS expansions are cumulative, while only actual inference consumes process quota. Applied acquire/install/reveal advance revision and state hash exactly once; applied dialogue and non-applied outcomes preserve state; `STALE` is rejected. Replay uses redacted recorded-intent traces with zero model calls; native replay reexecutes immutable input via `--npc-experiment` as `engine_fixture_reexecution`, never `--npc-replay`. UI cancellation invalidates local directives only; provider work may finish, and `core.cancel` no-publication is a unit/controller property, not a live-route claim.

C. Validate–Repair–Commit

Fig. 2 gives dependency-free pseudocode: "Record" appends in memory; persistence follows terminal consistency checks.

Repair receives structured error codes, never mutates state directly, and returns unchanged deterministic fallback on exhaustion (Fig. 1). Of the two pilot callbacks, the *typed validator-error-guided operator* (`counterexample_guided_repair`) reads only

$a \leftarrow \textsc{Propose}(c)$; $j \leftarrow 0$
repeat
  $(ok, E) \leftarrow V(c, a)$; $\textsc{Record}(a, E)$
  if ok **then**
    assert $V(c, a).ok$; **return** $\textsc{Commit}(c, a)$
  if $j = K$ **then return** $\textsc{Fallback}(c)$
  $a \leftarrow \textsc{Repair}(c, a, E)$; $j \leftarrow j + 1$
until terminal

Fig. 2. Validate–repair–commit procedure. At most $K+1$ candidate attempts are recorded; the successful path performs a defensive validation before mutation.

TABLE III
PER-ERROR-CLASS EDITS OF THE GUIDED OPERATOR ρ

Validator error class	Edit $\delta(e)$ on the candidate
Policy precondition omission	insert the named predicate
Policy effect omission	insert the named effect
Policy effect violation	drop the named effect
Unsatisfied precondition	drop the named predicate
Unauthorized stage mutation	clear `quest_stage_effect`
Unknown action type	no-op (no registered alternative)
Unreachable object; knowledge, disclosure, and stage classes	no-op (candidate-local edit would launder or needs state)

prior candidate a and typed errors E, its sole state-derived input, never authoritative `WorldState`:

$$\rho(a, E) = a \oplus \bigcup_{e \in E} \delta(e), \tag{3}$$

where each $\delta(e)$ is the deterministic per-error-class edit of Table III and $\oplus$ applies the resulting field edits. Each edit touches at most one candidate field per error entity, so the number of changed fields per attempt is bounded by $|E|$; applying ρ twice with the same error set is a fixed point only for that identical E, not a convergence claim across changing error sets or attempts; and an entity simultaneously demanded and rejected on the same field is declared irreparable and left untouched. The *state-reading reference callback* instead discards the error set and reconstructs the policy-governed fields (preconditions, effects, and both quest-stage fields) directly from the authoritative state; it deliberately never edits declaration-integrity fields (required objects, used facts, disclosed facts), because those are authored declarations rather than state-derived repairs.

D. Integrity, Semantic Replay, and Accounting

Each result row carries an unkeyed SHA-256 *integrity checksum*, and proposal outcomes link to a trace checksum; these detect accidental or tested content changes but are not signatures, writer identity, or protection against an actor who rewrites content and checksums. Semantic replay strictly parses the prior state and every recorded candidate/validation pair, requires every nonfinal attempt to be invalid, recomputes the terminal transition, compares the stated final state, and requires continuity between adjacent JSONL records; it does not re-execute the repair step that produced candidate $j+1$. Replay checks request→decision→result order, binds the first issued action to exactly one receipt, and claims neither a duplicate-action-ID issuance ban nor cryptographic authentication.

TABLE IV
EVIDENCE LANES AT A GLANCE: DESIGN, UNIT, COMPARISON, AND CEILING (GENERATED FROM THE EXPERIMENT MATRIX)

Lane	Design	Unit / budget	Comparison	Ceiling
E1	Offline conformance: authored frozen fixtures, one world	13 gate; 12 repair/arm; 10 faults; 7 adapter; 3 guards	reject vs blind vs ρ vs oracle	mechanism conformance
E2	Screening-pilot-only RQ2: one hosted proposer, matched candidate	5 cells $\times$ 5 calls; K=1	ρ vs blind per regime	screening-pilot-only transfer screen
E3	KG/ontology simulation: closed-world typed-link holdout	43 nodes; 106 ref. edges; 24 typed; 210 scores	degree baseline vs 6 fixed strategies	construction result only
ENG1	Engineering-only Godot/Web: engine-local policy mirror, no participants	4 fixtures; 52 checks; 8 smoke; 5 archetypes	authored fixture paths	engineering-only conformance

Lanes are non-interchangeable with different denominators; E2 is screening-pilot-only and ENG1 is engineering-only. None establishes population efficacy, model ranking, player experience, retrieval benefit, or commercial-engine performance.

Terminal classes are commit, symbolic fallback, adapter failure, and controller failure. Controller failure preserves state and its frozen assignment row but may lack a completed proposal trace when no candidate outcome exists; complete results do not require a trace per row. Pre-execution keys bind run, arm, scenario, seed, model, revision, controller-configuration hash, assignment-input hash, and prior-state hash. Aggregation rejects duplicate/missing/unexpected keys and separates symbolic, adapter, and controller failures.

V. EVIDENCE LANES AND BOUNDARIES

Table IV fixes lane designs, units, comparisons, and inference ceilings.

A. *E1* Offline Conformance Design

The pilot asks whether (PQ1) valid candidates commit while named invalid-policy cases remain unchanged; (PQ2) rejection-only, blind unchanged retry, ρ, and the state-reading reference callback produce the expected outcome in every frozen repairability class; (PQ3) frozen checksum, state-semantic, linkage, and continuity mutations are rejected by their designated check operations and an injected second-file append failure triggers the pair-write rollback check; (PQ4) strict adapters and assignment manifests retain exactly one classified terminal row per assigned case or fail closed; and (PQ5) proposal and replay parsers both reject an otherwise-valid candidate carrying one unknown top-level key.

Purposive conformance fixtures cover validator/state isolation, repair arms, record/trace fault injection, and adapter/accounting contracts; they are not population samples. Before execution, each repair fixture fixes its class and arm expectations: *guided-repairable* (error payload suffices for ρ), *oracle-only* (requires authoritative state), or *irreparable* (no arm may commit). Denominators count unique cases: 13 gate fixtures (one valid control, 12 isolated negatives) and 12 separate repair cases.

Measures are exact counts: validator-class agreement; rejected or failed cases with state mutation; commit and attempt outcomes by arm and repairability class; detectable

faults rejected by their designated operation; replayed traces; rejected manifest violations; and proposal/replay parity on the unknown-key regression, which is parser-contract parity rather than a safety outcome. Exception class alone does not identify a stable detector layer, so no detector attribution is reported, and no p-values or population intervals are attached. Adapter cases carry JSON-Schema-pinned synthetic latency and token constants that exercise accounting propagation only. The frozen offline packet binds fixtures, assignments, schemas, results with embedded traces, code revision, and checksums, following reproducibility and measurement-boundary guidance [38], [39].

B. *E2 Live RQ2 Screening*

E2 compares blind unchanged retry with ρ at $K{=}1$: five seed-indexed hosted `gpt-5.6-sol` calls per cell each supply one candidate to both arms, removing within-call proposal differences. Conditions vary policy visibility and trivial zero-effect-route availability. The pilot-only frozen packet retains current cells/superseded v1 diagnostics and SHA-256-binds receipts. Uncontrolled sampler seeds, unpinned revision, unavailable token accounting, and constructed nonpopulation states limit reporting to exact counts/failure classes, without p-values, confidence intervals, or model ranking.

C. *NPC-ENG1 Fixed-goal engineering lane*

NPC-ENG1's 16 deterministic native rule-control cases cover bootstrap, identity/hash binding, intent/plan/role/result transitions, negative gates, stale/duplicate/deadline paths, and replay. Stable v5 defines 16 engineering assignments/12 screening recipes; verified-run-002, verified-run-003, and verified-run-004 remain immutable historical packets. Historical preflight is outside NPC-SCR1. Only the validated `research/academic-pipeline/npc-goal-recovery/verified-run-006/` packet and repository runner/validator support final status/count/latency and reproduction: protocol/engine evidence only.

D. *NPC-SCR1 Descriptive screening*

NPC-SCR1 assigns 12 unique cases $\times$ two repetitions $\times$ three arms (P/R/O): 72 rows, 24/arm. NPC-ENG1 adds 16, giving 88 rows over 28 definitions ($R = 40$, $P = 24$, $O = 24$), separate from legacy ENG1/E1. All pre-trigger, parse, deadline/cancel, duplicate/stale, controller, and no-route rows for one game-owned Mira goal remain; recovery-eligible/unreachable strata stay separate. P is authenticated Codex OAuth without fallback; R is authored rule/grammar control; O is a protocol-declared same-family auxiliary using identical public input (family match unverified; no independent gold; auxiliary-source fraction 100%, independent fraction zero). Ownership and role/action/replanning gates follow Sec. IV; Mira's disclosure requires known `tide_marks_hint` and disclosure authorization. Screening establishes descriptive provenance, not recovery efficacy, semantic gold, accuracy ceilings, model ranking, or

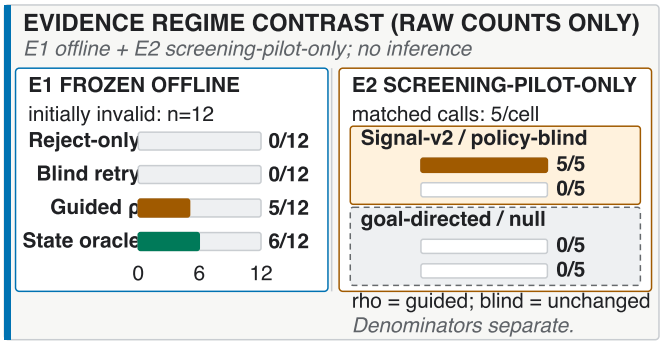


Fig. 3. Evidence-regime contrast using E1 frozen offline counts and E2 screening-pilot-only matched counts. E1 shows four arms over 12 designed invalid cases (0/12, 0/12, 5/12, 6/12). E2 separates Signal-v2/policy-blind (ρ 5/5 vs. blind 0/5) from the goal-directed/goal-blind null regime (0/5 vs. 0/5). Counts are descriptive; denominators are not pooled and support no efficacy, population, model-ranking, or player-experience inference.

C6. Historical source-bound `native012` separately demonstrates Godot→HTTP→authenticated Codex NPC v1.1 (one intent, three observations, separate zero-model-call replay), outside legacy ENG1 and the 16+72 pool; it does not revalidate post-fix runtime. Legacy ENG1 remains source-bound Godot authored-trace replay. Generated P/R/O results are observed rows; run-006 P lacks requested/returned model labels and provider revision. Normal-play full-loop episodes are outside this one-step pool. `aggregate.validation` is a generation-time placeholder, not admissibility: only `validation-report.json` plus fresh strict validation establish that.

E. *E3 KG/Ontology Link-Proposal Simulation*

The third lane is an engineering-only closed-world simulation, separate from the authoritative `WorldState`: 43 graph nodes and 106 reference edges, 24 reviewed typed edges, passing type/domain/range checks and six competency questions, with six graph-edge-withheld authored targets used both to select and summarize the retained strategy—not an untouched or external test set—and scored under seven fixed strategies. The retained typed-lexical strategy (S2) recovered 6/6 authored targets ($P = R = F_1 = 1.000$, realistic-tie $\mathrm{MRR} = 0.944$, Brier $= 0.131$, $\mathrm{Sem@3} = 1.000$); the degree baseline recovered none. These are construction results: not OWL/SHACL conformance, semantic completeness, runtime retrieval, or evidence of any efficacy claim.

VI. Pilot Results

A. *Declared-Field Gate Conformance*

The pilot loader admits a fixture set only when it isolates every implemented validator code exactly once alongside one valid control, so the 13/13 agreement and the observation of all 12 implemented error codes are *construction invariants of the harness*, not measured success rates. What the run adds is the part the loader does not enforce: each of the 12 isolated negative fixtures showed observed agreement with its authored expected code rather than reaching a different one, and every

TABLE V
E1 OFFLINE CONFORMANCE: EXACT REPAIR-ARM COUNTS BY REPAIRABILITY CLASS (DESIGNED FIXTURES)

Arm	Info source	Commits / cases			Mean edits[e]
		G-rep.[f]	O-only[f]	Irrep.[f]	Fields
Rejection-only (K=0)	none	0/5	0/1	0/6	—
Blind unchanged retry	none	0/5	0/1	0/6	—
Validator-error-guided ρ	error set E	5/5	0/1	0/6	1.0
State-reading oracle	state+policy	5/5	1/1	0/6	1.0

[e]Mean changed candidate fields over committed repairs (minimality proxy); failed arm executions left the state unchanged. E1 uses 12 initially invalid designed cases: rejection-only/blind 0/12, 0/12; validator-error-guided ρ 5/12; state-reading oracle 6/12 commits. [f]Commits/cases. G-rep. = guided-repairable, O-only = oracle-only, Irrep. = irreparable (frozen repairability classes); these are not population efficacy or live-model quality.

TABLE VI
E1 OFFLINE CONFORMANCE: FROZEN-PILOT CHECKS AND TERMINAL OUTCOMES (ROWS ARE NOT COMPARABLE)

Check or terminal family	Exact outcome	Interpretation
Gate fixture agreement	13/13	authored-oracle agreement
Known-boundary acceptance	2/2	encoded limit, not a safety pass
Closed unknown-key rejection	1/1	proposal/replay parser parity
Detectable integrity-fault rejection	10/10	designated check operations
Known provenance-boundary acceptance	1/1	repair provenance unauthenticated
Adapter terminal classes	1 commit; 1 fallback; 5 failures / 7 assigned	complete classification of 7 assignments
Manifest-guard rejection	3/3	injected faults failed closed

Rows have different denominators and favorable directions; they must not be ranked as rates. Every value is an exact audit fact for designed fixtures.

noncommit path preserved the complete canonical state. This is implementation conformance to an authored structured-field oracle over a single world state, not accuracy against independent semantic labels.

B. Repair-Arm Comparison on Designed Fixtures

The 12 initially invalid designed cases are partitioned into frozen repairability classes: 5 guided-repairable, 1 oracle-only, and 6 irreparable. Rejection-only and blind unchanged-candidate retry each committed 0/12. The validator-error-guided operator ρ committed 5/5 guided-repairable cases and, by design, 0/1 oracle-only and 0/6 irreparable cases; every ρ commit edited a mean of 1.0 candidate fields. ρ reads only the prior candidate and typed validator errors E; those errors are its sole state-derived input, and every failed arm execution left the canonical state unchanged. The state-reading reference callback committed 5/5 guided-repairable and 1/1 oracle-only cases, with 0/6 irreparable. These exact counts separate the three mechanisms on the designed fixtures: blind retry never converts a validator error into a commit, ρ converts exactly the cases whose error payload carries sufficient information, and the oracle additionally converts the case requiring state knowledge. The 5 committed guided cases initially carried `POLICY_PRECONDITION_OMISSION+UNSATISFIED_PRECONDITION`, `POLICY_EFFECT_OMISSION`, `POLICY_EFFECT_VIOLATION`, `UNSATISFIED_PRECONDITION`, `POLICY_QUEST_STAGE_EFFECT_VIOLATION`, respectively, and each became valid through the corresponding Table III edit $\delta(e)$ alone. This is operator conformance on frozen fixtures, not repair quality of any live model.

C. Integrity, Boundaries, and Assignment Accounting

All 10 faults prespecified as detectable were rejected by their designated check operations (10/10). Because this pilot does not compare stable typed detector codes, it does not establish detector-layer attribution. Separately, the known provenance-boundary fixture substituted a different invalid precursor before the same recorded valid repair, recomputed the checksum, and passed replay as expected (1/1 undetected). Thus, the mechanism does not protect against a writer who can recompute unkeyed checksums, and replay does not authenticate the repair-generation operation. Two open boundary sentinels—unextracted narrative disclosure and simultaneous omission of a required object from candidate and policy—were intentionally accepted at the encoded layer (2/2), with 0 labelled as safety passes. The former unknown-top-level-field acceptance boundary was later reclassified as a closed negative regression: both proposal and replay parsers specifically rejected a complete 12-field candidate carrying one unknown key (1/1). This establishes parser rejection parity, not semantic safety. Among seven adapter/accounting assignments, outcomes were 1 commit, 1 symbolic fallback, and 5 adapter failures. Synthetic telemetry fields were populated and propagated for 2/7 assignments; these are schema-pinned constants verifying accounting-field propagation, not measurements. All 3/3 injected duplicate or missing-assignment guards failed closed. These are raw counts from synthetic offline fixtures.

D. Live Screening Pilot (Pilot-Only)

A separate screening pilot called the hosted proposer gpt-5.6-sol. It used 5 seed-indexed proposal calls per cell and repair budget $K = 1$; within each call, blind unchanged retry and ρ received the same initial candidate. The hosted revision was unpinned, token accounting was unavailable, and the seeds did not control the hosted sampler, so the calls are quasi-replicates rather than independent randomized samples. We report neither inferential statistics nor a model ranking.

In Table VII, a guided advantage appeared only in the policy-blind cell of the signal-repair state (Signal-v2), a state deliberately constructed to elicit guided-repairable errors. All 5/5 initial candidates were invalid; ρ committed 5/5, whereas blind retry committed 0/5. The other 4 current cells showed no guided advantage: 3 had already-valid initial candidates, and the frozen-base goal-directed cell had QSR in 5/5, a null regime with guided and blind commits of 0/5 and 0/5. All 15/15 noncommit arm outcomes in the current cells preserved the prior-state hash. This supports mechanism transfer only in that error regime, not population efficacy, model promotion,

TABLE VII
E2 SCREENING-PILOT-ONLY: LIVE SCREENING EXACT COUNTS (5 CELLS, 5 PROPOSALS PER CELL, K=1)

Base / condition	Initial valid	ρ vs blind commits	Δ	Initial errors
Frozen / visible	5/5	5/5 vs 5/5	+0	—
Frozen / blind	5/5	5/5 vs 5/5	+0	—
Frozen / goal-blind	0/5	0/5 vs 0/5	+0	QSR
Signal-v2 / visible	5/5	5/5 vs 5/5	+0	—
Signal-v2 / blind	0/5	**5/5 vs 0/5**	**+5**	PEO+PEV+PPO

QSR = quest-stage regression, PEO = policy-effect omission, PEV = policy-effect violation, PPO = policy-precondition omission. Δ is ρ commits minus blind commits for matched candidates. There are 5 cells with 5 calls each: 3 cells were initially valid (5/5), while the goal-blind cell had QSR in 5/5 and 0/5 and 0/5 commits in both arms (null regime). Only Signal-v2/blind separated (ρ 5/5 vs blind 0/5). All counts are screening-pilot-only, with no inferential statistics or model ranking.

or a general sample-efficiency claim. Accordingly, the two screening observations are pilot-only; the confirmatory transfer claim remains open. In the superseded v1 diagnostic, guided repair reduced up to 3 co-occurring errors to one irreparable knowledge error but still could not commit, showing that partial per-error repairs do not compose when any residual error is irreparable.

E. Fixed-Goal NPC Operation (NPC-ENG1/NPC-SCR1)

Mira's fixed goal is to deliver the authorized tide lead after harbor-signal restoration. Python computes bounded BFS plans from typed intent and public observations. Acquire/install remain player actions; the NPC may only commit the authorized `reveal_hint`. Godot synchronously checks session/epoch/request/goal/policy/revision/hash before calling the existing writer without an await. Results and new observations determine the next plan, unlike single-candidate validator-error repair.

NPC-ENG1 comprises 16 deterministic R-arm engineering cases initialized from actual engine snapshots (assigned_failure=2, completed=14). NPC-SCR1 retains all 72 assignments and failures, 24 per P/R/O arm (assigned_failure=12, completed=60). P records 24 authenticated Codex CLI exec attempts, 24 returned decisions, and 4 assigned failures; these are not remote billing-call counts. Requested model: unknown (null); recorded decision-model identifier: unknown (null). The latter propagates the configured selector, not a provider revision; the provider revision is unavailable. Null identifiers remain unknown; global defaults are not evidence. R uses authored rules. O is a protocol-declared auxiliary under `same_model_family_auxiliary`. Actual model-family matching is unverified. No source is independent; O is neither an oracle nor semantic gold. Table VIII and Fig. 4 report bounded protocol-conformance fixed-case counts, not recovery-efficacy, superiority, or generalization estimates. Efficacy remains pending; tokens and cost are unmeasured. A read-only CLI and isolated working directory were used, but tool-free inference and adversarial confidentiality were not established. Verified pre-authored stale-control ACK rejections are P=4, R=4, O=4 per arm (SCR-T01 (stale_revision), SCR-T02 (stale_policy)); 12 instances had all corresponding negative-control gates passed. These rows

TABLE VIII
NPC-SCR1 FIXED-CASE EXECUTION COUNTS, SEPARATE FROM THE 16 NPC-ENG1 CASES; ALL ASSIGNED FAILURES REMAIN IN THE DENOMINATOR.

Arm	Assigned	Complete/fail	Goal reached	Recovery E/U
P	24	20/4	4	24/0
R	24	20/4	4	24/0
O	24	20/4	4	24/0

E: recovery eligible; U: unreachable; NA: unmeasured. Observed goal counts are not recovery rates. Valid replay does not turn a failed assignment into a completed one. O has zero independent sources.

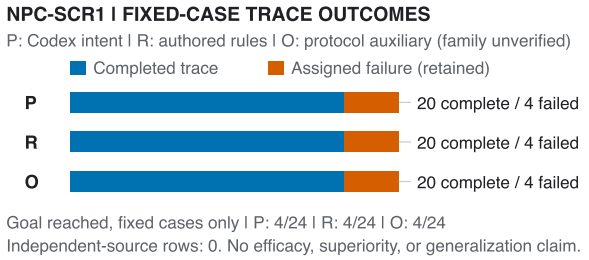


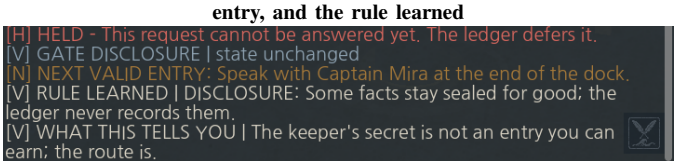
Fig. 4. NPC-SCR1: $12 \times 2 \times 3 = 72$ assigned rows (24/arm); NPC-ENG1 adds 16, totaling 88 over 28 definitions ($R/P/O = 40/24/24$), separate from legacy ENG1/E1 and the full-loop demo. P: authenticated Codex intent; R: authored rules; O: protocol-declared auxiliary (family unverified; no independent gold/rows). ENG-X01/SCR-X01 and v1–v4 remain provenance. Counts are descriptive, not recovery efficacy, oracle comparison, normal-play, or player benefit.

remain assigned failures in the denominator, not method-performance outcomes; other assigned failures (0) remain unclassified. Replay validation is limited to the exported epoch (`intent_lineage_scope=epoch_local_only`); it does not prove intent continuity across epochs (`cross_epoch_intent_continuity_proven=false`). The opaque-anchor exception is restricted to an adjacent leading observation request/decision pair in a positive-epoch snapshot. Epoch-zero orphans, other observation requests before intent commitment, and later intent mismatches remain rejected.

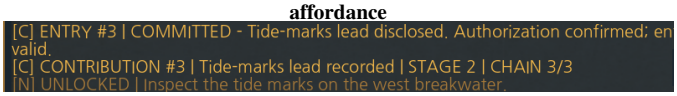
Tables V and VI report exact designed-fixture counts: 13/13 gate agreement is a loader invariant, each repair arm has 12 unique cases ($5 + 1 + 6$), and five live cells contribute 25 proposal calls outside the offline denominator; neither supports population or live-model estimation. Only designated fixture checks admit offline rows; the separately hash-bound live packet never enters that denominator (Table IV).

ENG1. (Legacy; no live transport.) Godot 4.7.1 replays an authored trace: forbidden disclosure holds the prior hash; authorized commits reach the offline terminal hash (Fig. 5). Evidence is the trace/deterministic evaluation, not pixels. Committed facts populate the case-file board, hints, coach, and chain prompt; the fail-closed dashboard distinguishes checks, holds, unreached in-scope checks, and out-of-scope families. Its 52/52 core and four extension checks establish engine-local conformance only: no live model/Python transport, participants, narrative result, usability, or efficacy.

(a) Held proposal: the gate family, the unchanged state, the next valid entry, and the rule learned



(b) Committed proposal: entry, contribution clause, and unlocked affordance



(c) End-card receipt after the three commits

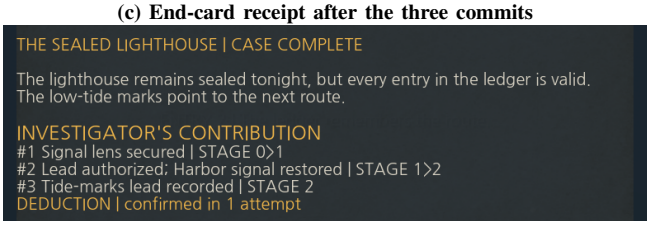


Fig. 5. Presentation-only Godot ledger crops: three scripted stages of one authored fixture (1280×720 engineering captures; illustration, not evidence). (a) Held disclosure: named gate family, unchanged state, next valid entry, reusable rule. (b) Commit: entry, contribution from committed-snapshot delta only (added facts, quest $1>2$, chain $2/3$), unlocked affordance. (c) End card: three contributions, hold count, validator receipt and replay terminal hash (`4b2310...`). These committed-snapshot readouts (Fig. 1, lower strip) use no oracle labels, hidden facts, live model, or participants and support no usability, immersion, affect, efficacy, G4, or G6 claim.

VII. Discussion

A. Established Scope

Results are mechanism-only: checks isolate state; declared repair/fallback paths hold; named corruptions reject; rows remain classified; no stable detector attribution. NPC ownership, boundaries, and descriptive-only count ceilings are fixed in Sec. V; they exclude efficacy, superiority, generalization, human benefit, cost equality, semantic gold, and accuracy ceilings.

State-reading handles oracle-only cases that ρ cannot handle; typed errors let ρ match it only on guided-repairable cases. Live blind-retry separation occurred only there; already-valid/irreparable null regimes, unpinned revision, and quasi-replicated calls preclude sample-efficiency/model-capability claims. Self-Refine, Reflexion, and Self-Debugging are precedents [48], [55], [56]; ρ instead consumes deterministic pre-execution errors and isolates failed arms, but is not semantic repair, only analogous to counterexample-guided synthesis [57].

B. Relationship to Prior Work

TRACE-RPG complements rather than outperforms constrained decoding, retrieval/memory, executable benchmarks, and process-linked measurement. Planned, not executed: preregistered multi-model/oracle/blind-retry comparison, matched-validity retrieval/memory ablations, and a blinded matched-validity human study with justified power/mixed-effects; screening replaces none [40], [37], [45].

VIII. Threats, Ethics, and Artifact Scope

Nine boundaries apply: **L1:** encoded fields miss semantics; consistency is not behavior consistency [58], and narrative quality is unmeasured. **L2:** one world/live state gives no cross-world, cross-policy, or scale generalization. **L3:** state-reading is an oracle; classes/regimes do not establish repair quality/sample efficiency. **L4:** one hosted proposer has unpinned revision, no token accounting, and uncontrolled seeds; E2 screening has no Python–Godot authorization round trip. **L5:** no participant/affect data or comprehension/agency/enjoyment/immersion/benefit evidence. **L6:** unkeyed hashes provide integrity, not authentication/tamper resistance. **L7:** one-writer consistency leaves concurrency unverified. **L8:** one Godot fixture has startup-dominated samples above 16.7 ms; portability/performance are untested. **L9:** fixtures/cells support no population, safety-rate, or model-ranking inference. Internal validity requires independent policy expectations/no leakage; reproducibility requires pinned software/artifacts. NPC deadlines, duplicate/stale/late-drop handling, cumulative budgets, and synchronous bindings are protocol invariants, not latency/quality measurements. Sec. IV fixes local-only cancellation/provider-completion and unit/controller-only `core.cancel` boundaries, predicates, and player requests; none establishes human, cross-world, independent-gold, or C6 efficacy evidence.

No participants or personal telemetry were collected. Planned human/affect work requires ethics review, consent, compensation disclosure, minimization, retention/deletion, and opt-out; LLM judges remain auxiliary and require calibration/bias controls [40], [41], [42], [43].

IX. Conclusion

TRACE-RPG gates mutation and audits linked outcomes/replay. Frozen fixtures show conformance; pilot-only screening separates ρ from blind retry (5/5 vs. 0/5) only in the constructed repairable regime. Godot readouts expose committed-snapshot rules/contributions; cross-model/player/retrieval/memory/affect/commercial benefits remain untested. Sec. V separates additive NPC, historical `native012` (not post-fix revalidation), and legacy ENG1 evidence: contract/provenance, not production-readiness, generalization, efficacy, or player benefit.

Disclosure of AI Usage

LLMs assisted prose, code, test drafting, and citation-identity checks; a hosted model supplied labelled live-screening candidates. Counts come from deterministic runners or hash-bound receipts. UI art (portrait, seals, panel textures, case-file board, evidence icons, guide vignette) and the evidence-desk mesh are curated AI-generated assets with per-file provenance; Fig. 5 state values, hashes, and ledger lines are engine-rendered from committed snapshots. Authors verified claims against source artifacts and accept full responsibility. The additive NPC tables and figure are generated separately from the NPC-GOAL-RECOVERY packet, preserve all assigned rows, bind project/protocol/schema hashes, and record unavailable token/cost/model-revision fields as null, not zero.

Data and Code Availability

The repository provides implementation/build sources, frozen offline/live packets, the Godot slice and deterministic evaluation, and machine-checked contribution, reference-topic, and experiment matrices generating Tables I and IV; no reviewer archive or DOI deposit is claimed.

References

[1] M.-A. Côté, Á. Kádár, X. Yuan *et al.*, "TextWorld: A learning environment for text-based games," in *Computer Games*, ser. Communications in Computer and Information Science. Springer International Publishing, 2019, vol. 1017, pp. 41–75.

[2] R. Wang, P. Jansen, M.-A. Côté, and P. Ammanabrolu, "ScienceWorld: Is your agent smarter than a 5th grader?" in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2022, pp. 11 279–11 298.

[3] N. Weir, R. Thomas, R. d'Amore, K. Hill, B. Van Durme, and H. Jhamtani, "Ontologically faithful generation of non-player character dialogues," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 9212–9242.

[4] T. Ashby, B. K. Webb, G. Knapp, J. Searle, and N. Fulda, "Personalized quest and dialogue generation in role-playing games: A knowledge graph- and language model-based approach," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, 2023, pp. 1–20.

[5] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without finetuning," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 10 932–10 952.

[6] L. Beurer-Kellner, M. Fischer, and M. Vechev, "Prompting is programming: A query language for large language models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1946–1969, 2023.

[7] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021, pp. 9895–9901.

[8] X. He, Y. Tian, Y. Sun *et al.*, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in *Advances in Neural Information Processing Systems*, vol. 37. Neural Information Processing Systems Foundation, 2024, pp. 132 876–132 907.

[9] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems*, vol. 37. Neural Information Processing Systems Foundation, 2024, pp. 59 532–59 569.

[10] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in *ECAI 2025*, ser. Frontiers in Artificial Intelligence and Applications, vol. 413. IOS Press, 2025, pp. 2993–3000.

[11] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 2023, pp. 1–22.

[12] Y. Shao, L. Li, J. Dai, and X. Qiu, "Character-LLM: A trainable agent for role-playing," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 13 153–13 187.

[13] N. Akoury, Q. Yang, and M. Iyyer, "A framework for exploring player perceptions of LLM-generated dialogue in commercial video games," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 2295–2311.

[14] M. Hochreiter, S. Kriglstein, and G. Wallner, "Beyond pre-defined scripts: Player perceptions on generative non-player character dialogues," in *Proceedings of the 31st International Conference on Intelligent User Interfaces*. ACM, 2026, pp. 2004–2018.

[15] M. Yin, H. Zu, W. Gu *et al.*, "How contextualized generative AI shapes player experience in games," *Entertainment Computing*, vol. 58, p. 101194, 2026.

[16] V. Figueiredo and D. Elumeze, "Symbolically scaffolded play: Designing role-sensitive prompts for generative NPC dialogue," 2025, preprint.

[17] G. N. Yannakakis and J. Togelius, "Experience-driven procedural content generation," *IEEE Transactions on Affective Computing*, vol. 2, no. 3, pp. 147–161, 2011.

[18] D. Melhart, D. Gravina, and G. N. Yannakakis, "Moment-to-moment engagement prediction through the eyes of the observer: PUBG streaming on Twitch," in *FDG '20: Proceedings of the 15th International Conference on the Foundations of Digital Games*. ACM, 2020, pp. 1–10.

[19] Z. Wang, Q. Guo, R. Singh, and X. Hu, "Do vision language models understand human engagement in games?" 2026, preprint.

[20] L. Fan, G. Wang, Y. Jiang *et al.*, "MineDojo: Building open-ended embodied agents with internet-scale knowledge," in *Advances in Neural Information Processing Systems*, vol. 35. Neural Information Processing Systems Foundation, 2022, pp. 18 343–18 362.

[21] D. Pérez-Liébana, J. Liu, A. Khalifa, R. D. Gaina, J. Togelius, and S. M. Lucas, "General video game AI: A multitrack framework for evaluating agents, games, and content generation algorithms," *IEEE Transactions on Games*, vol. 11, no. 3, pp. 195–214, 2019.

[22] X. Liu, H. Yu, H. Zhang *et al.*, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations*, 2024, pp. 52 989–53 046.

[23] C. Ma, J. Zhang, Z. Zhu *et al.*, "AgentBoard: An analytical evaluation board of multi-turn LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37. Neural Information Processing Systems Foundation, 2024, pp. 74 325–74 362.

[24] X. Zhou, H. Zhu, L. Mathur *et al.*, "SOTOPIA: Interactive evaluation for social intelligence in language agents," in *International Conference on Learning Representations*, 2024, pp. 40 975–41 019.

[25] D. Paglieri, B. Cupiał, S. Coward *et al.*, "BALROG: Benchmarking agentic LLM and VLM reasoning on games," in *International Conference on Learning Representations*, 2025, pp. 96 666–96 702.

[26] P. Le Pelletier de Woillemont, R. Labory, and V. Corruble, "Automated play-testing through RL based human-like play-styles generation," *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 18, no. 1, pp. 146–154, 2022.

[27] Y. Ma, J. Yan, B. Shi *et al.*, "Can LLM agents stick to the script? a benchmark for long-horizon consistency in interactive narratives," 2026, preprint (v1); ICML 2026 acceptance comment on arXiv, proceedings record unverified.

[28] S. Kambhampati, K. Valmeekam, L. Guan *et al.*, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 22 895–22 907.

[29] B. Ichter, A. Brohan, Y. Chebotar *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proceedings of the 6th Conference on Robot Learning (CoRL 2022)*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–318.

[30] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," 2025, preprint (v3); ICSE 2026 acceptance stated on arXiv, proceedings record unverified.

[31] Z. Zeng, S. Li, J. Xi, A. Zhu, and P. Ammanabrolu, "Setting the DC: Tool-grounded D&D simulations to test LLM agents," in *NeurIPS 2025 Workshop on Generative and Protective AI for Content Creation (GenProCC)*, 2025, non-archival workshop paper; OpenReview/PDF list Zeng first, with Zeng and Li as equal contributors; NeurIPS/GenProCC lists display Li first.

[32] E. Zhou, S. Basavatia, M. Siam, Z. Chen, and M. O. Riedl, "STORY2GAME: Generating (almost) everything in an interactive fiction game," 2025, preprint.

[33] S. Buongiorno, L. Klinkert, Z. Zhuang, T. Chawla, and C. Clark, "PANGeA: Procedural artificial narrative using generative AI for turn-based, role-playing video games," *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 20, no. 1, pp. 156–166, 2024.

[34] M. Vaucher, S. Silveira, S. Góngora, and L. Chiruzzo, "IVIE: A neuro-symbolic approach to incremental and validated generation of interactive fiction worlds," in *Proceedings of the 17th International Conference on Computational Creativity (ICCC'26)*. Association for Computational Creativity, 2026, archival proceedings (ISBN); preprint arXiv:2606.13348.

[35] S. Góngora, L. Chiruzzo, G. Méndez, and P. Gervás, "World-state transformations for neuro-symbolic interactive storytelling," 2026, preprint.

[36] S. Geng, H. Cooper, M. Moskal *et al.*, "JSONSchemaBench: A rigorous benchmark of structured outputs for language models," 2025, preprint (v3).

[37] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, vol. 34. Curran Associates, Inc., 2021, pp. 29 304–29 320.

[38] J. Pineau, P. Vincent-Lamarre, K. Sinha *et al.*, "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[39] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the systematic reporting of the energy and carbon footprints of machine learning," *Journal of Machine Learning Research*, vol. 21, no. 248, pp. 1–43, 2020.

[40] C. van der Lee, A. Gatt, E. van Miltenburg, S. Wubben, and E. Krahmer, "Best practices for the human evaluation of automatically generated text," in *Proceedings of the 12th International Conference on Natural Language Generation*. Association for Computational Linguistics, 2019, pp. 355–368.

[41] M. Karpinska, N. Akoury, and M. Iyyer, "The perils of using Mechanical Turk to evaluate open-ended text generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021, pp. 1265–1285.

[42] L. Zheng, W.-L. Chiang, Y. Sheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36. Neural Information Processing Systems Foundation, 2023, pp. 46 595–46 623.

[43] G. H. Chen, S. Chen, Z. Liu, F. Jiang, and B. Wang, "Humans or LLMs as the judge? a study on judgement bias," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 8301–8327.

[44] D. Lakens, A. M. Scheel, and P. M. Isager, "Equivalence testing for psychological research: A tutorial," *Advances in Methods and Practices in Psychological Science*, vol. 1, no. 2, pp. 259–269, 2018.

[45] D. J. Barr, R. Levy, C. Scheepers, and H. J. Tily, "Random effects structure for confirmatory hypothesis testing: Keep it maximal," *Journal of Memory and Language*, vol. 68, no. 3, pp. 255–278, 2013.

[46] E. Feng, W. Zhou, Z. Liu *et al.*, "Get experience from practice: LLM agents with record & replay," 2025, preprint.

[47] Y. Sun, H. Wang, D. Li, G. Wang, and H. Zhang, "The emperor's new clothes in benchmarking? a rigorous examination of mitigation strategies for LLM benchmark data contamination," 2025, preprint (v1).

[48] A. Madaan, N. Tandon, P. Gupta *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 46 534–46 594.

[49] M. O. Riedl, C. J. Saretto, and R. M. Young, "Managing interaction between users and agents in a multi-agent storytelling environment," in *Proceedings of the Second International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2003, pp. 741–748.

[50] R. E. Fikes and N. J. Nilsson, "STRIPS: A new approach to the application of theorem proving to problem solving," *Artificial Intelligence*, vol. 2, no. 3–4, pp. 189–208, 1971.

[51] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.

[52] T. Haerder and A. Reuter, "Principles of transaction-oriented database recovery," *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, 1983.

[53] R. Evans and E. Short, "Versu—a simulationist storytelling system," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 6, no. 2, pp. 113–130, 2014, predecessor title of IEEE Transactions on Games.

[54] S. G. Ware and C. Siler, "Sabre: A narrative planner supporting intention and deep theory of mind," in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, vol. 17, no. 1, 2021, pp. 99–106.

[55] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 8634–8652.

[56] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024, accepted poster; OpenReview archival record; preprint arXiv:2304.05128.

[57] A. Solar-Lezama, L. Tancau, R. Bodík, S. Seshia, and V. Saraswat, "Combinatorial sketching for finite programs," in *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2006, pp. 404–415.

[58] Y. Huang, G. Chen, J. Yao *et al.*, "Beyond state consistency: Behavior consistency in text-based world models," 2026, preprint.